

<HLS Programming with FPGAs>

The HLS Book

Chapter 2

Finite Impulse Response (FIR) Filters 1

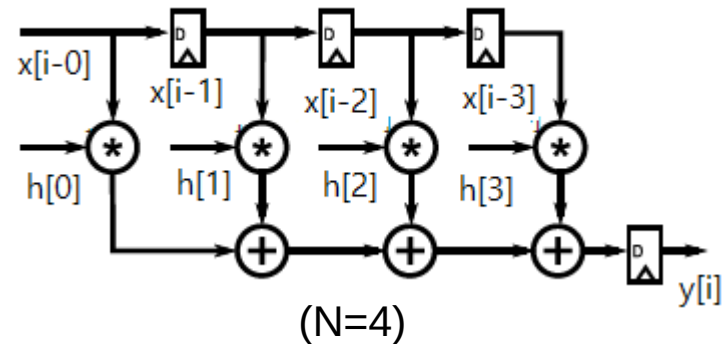
Choi, Young-kyu

Learning Objectives

- Understand the computation pattern of finite impulse response (FIR) filter
- Make a working, synthesizable HLS code for FIR
- Run a C/RTL simulation of your FIR code on Vitis HLS GUI
- Analyze the performance of your HLS design

Finite Impulse Response (FIR) filter

$$y[i] = \sum_{j=0}^{N-1} h[j] \cdot x[i - j]$$

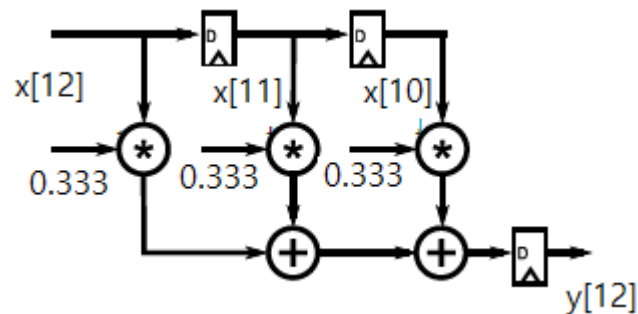


- One of the most widely used application in digital signal processing (DSP) domain
- Usage:
 - Signal restoration
 - Signal separation
- Convolution of coefficients $h[]$ and input signal $x[]$ and generates output signal $y[]$

Example: Moving Average Filter

$$y[i] = \frac{1}{N} \sum_{j=0}^{N-1} x[i-j]$$

(e.g. when $N=3$, $i = 12$) $y[12] = \frac{1}{3} \cdot (x[12] + x[11] + x[10])$



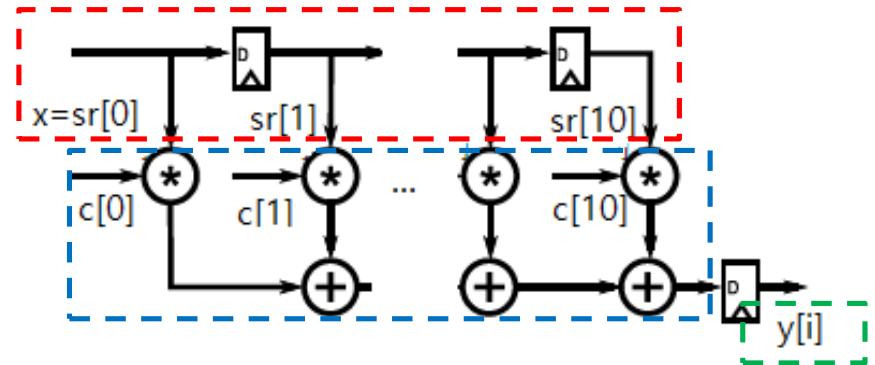
3 multipliers and 2 adders working in parallel

Basic HLS Code for FIR Filter

```
#define N 11  —————> 11 taps filter
#include "ap_int.h"
```

```
typedef int coef_t;
typedef int data_t;
typedef int acc_t;
```

```
void fir(data_t *y, data_t x) {
    coef_t c[N] = {
        53, 0, -91, 0, 313, 500, 313, 0, -91, 0, 53
    };
    static
    data_t shift_reg[N];
    acc_t acc;
    int i;
```



```
    acc = 0;
    Shift_Accum_Loop:
    for (i = N - 1; i >= 0; i--) {
        if (i == 0) {
            acc += x * c[0];
            shift_reg[0] = x;
        } else {
            shift_reg[i] = shift_reg[i - 1];
            acc += shift_reg[i] * c[i];
        }
    }
    *y = acc;
```

<from Figure 2.1 of HLS book>

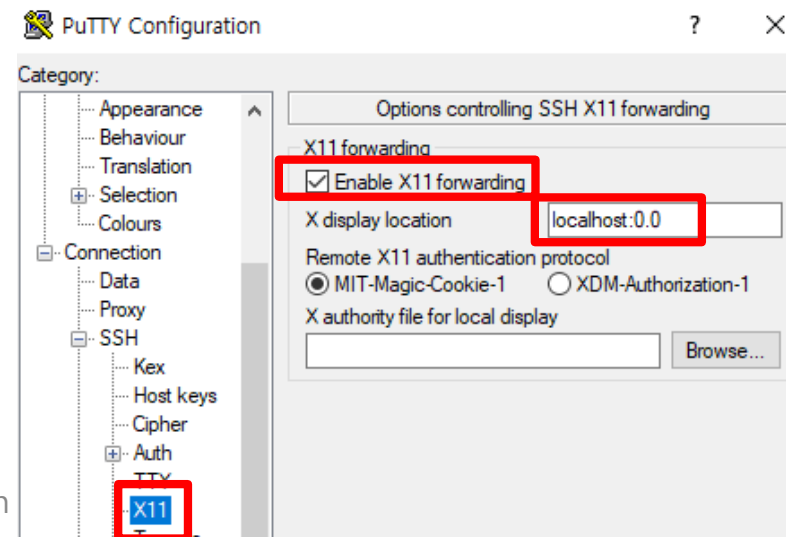
Launching Vitis HLS to Synthesize FIR code

- We need to launch Vitis HLS tool to synthesize the FIR code
- Two ways to launch Vitis HLS
 - Command line mode : Similar to the method used to synthesize the “hello_world” project
 - GUI mode : To be explained in the next slide
- More details on different execution modes will be later presented in Part “Vitis and Vitis HLS Flows”

Vitis HLS GUI mode on AWS

- Please use the AWS server nearest to your location
- Method 1) Use X (Windows)
 - On AWS:
 - Install any required dependencies:
 - Ex) `sudo yum install xorg-x11-server-Xorg xorg-x11-xauth xorg-x11-apps webkitgtk3`
 - Set environment variable: `export DISPLAY=localhost:0.0`
 - On your PC:
 - Enable X on Putty
 - Install Xming
- Method 2) Use NICE DCV
 - Recommended by AWS

(<https://sourceforge.net/projects/xming/>)



Vitis GUI Development with NICE DCV

- Follow the steps on: https://github.com/aws/aws-fpga/blob/master/developer_resources/DCV.md)
- More details for Step 1:

The screenshot shows the AWS Management Console interface for an EC2 instance. The instance is in a 'Running' state. A context menu is open for the instance, showing various actions. The 'Security' option is highlighted, and a red arrow points from the 'Modify IAM role' option to the 'Create new IAM role' button.

Instance state	Instance type	Status check	Alarm status	Availability
Terminated	m5.xlarge	-	No alarms	us-west-
Running	m5.xlarge	2/2 checks passed	No alarms	us-west-
Terminated	m5.xlarge	-	No alarms	us-west-

Actions available in the context menu:

- Launch instances
- Launch instance from template
- Connect
- Stop instance
- Start instance
- Reboot instance
- Hibernate instance
- Terminate instance
- Instance settings
- Networking
- Security
- Image and templates
- Monitor and troubleshoot

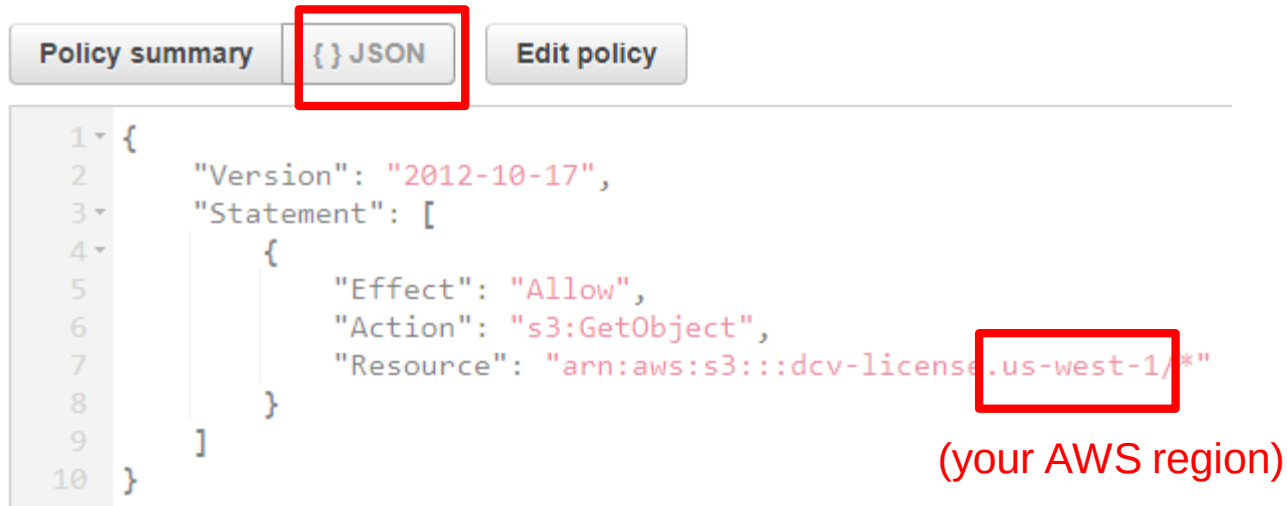
Additional options in the 'Security' submenu:

- Change security groups
- Get Windows password
- Modify IAM role

Buttons and links:

- Choose IAM role
- Create new IAM role

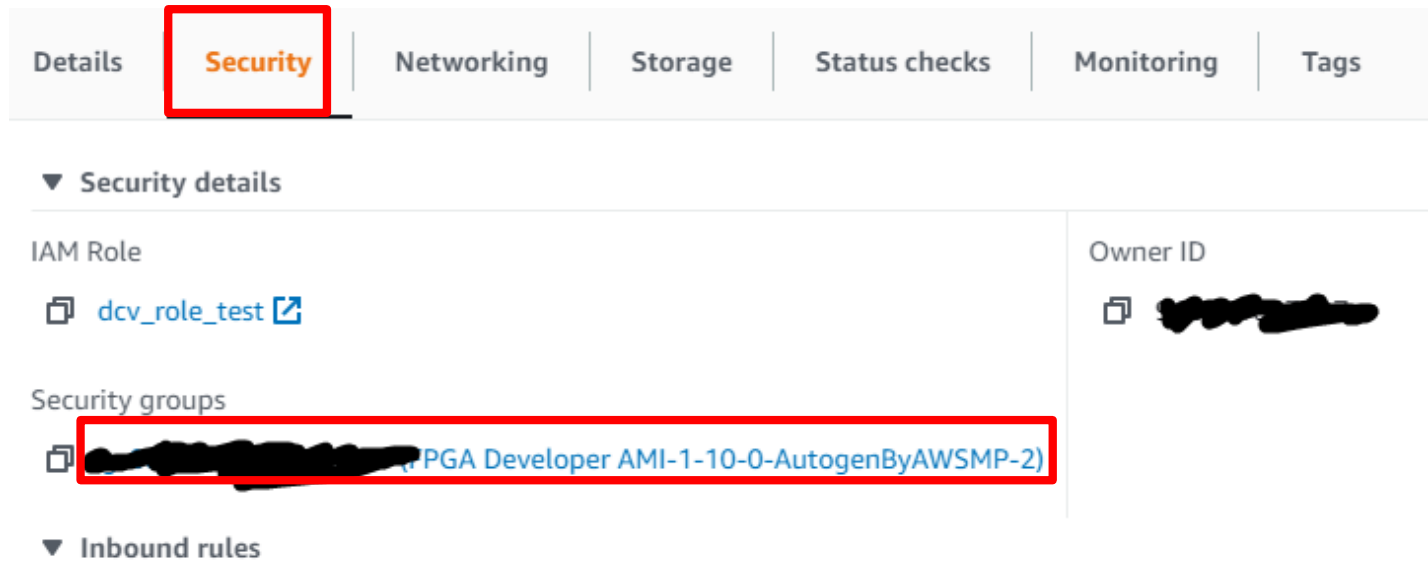
- More details for Step 1 (continued):



```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "s3:GetObject",
7       "Resource": "arn:aws:s3:::dcv-license.us-west-1/*"
8     }
9   ]
10 }
```

(your AWS region)

- More details for Step 2:



Details **Security** Networking Storage Status checks Monitoring Tags

▼ Security details

IAM Role
dcv_role_test

Owner ID
[REDACTED]

Security groups
sg-0123456789abcdef0 (PGA Developer AMI-1-10-0-AutogenByAWSMP-2)

▼ Inbound rules

- More details for Step 2 (continued):

Inbound rules (1/1)

Filter security group rules

< 1 > ⚙

Manage tags

Edit inbound rules

☒	Name ▾	Security group rule... ▾	IP version ▾	Type ▾	Protocol ▾	Port range ▾	Source
☒	–	ssh-████████████████████	IPv4	SSH	TCP	22	0.0.0.0/0

Inbound rules Info

Security group rule ID	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
ssh-████████████████████	SSH ▾	TCP	22	Custom ▾	0.0.0.0/0 ✕	Delete
–	Custom TCP ▾	TCP	8443	Anywhere-IPv4 ▾	0.0.0.0/0 ✕	Delete

Add rule

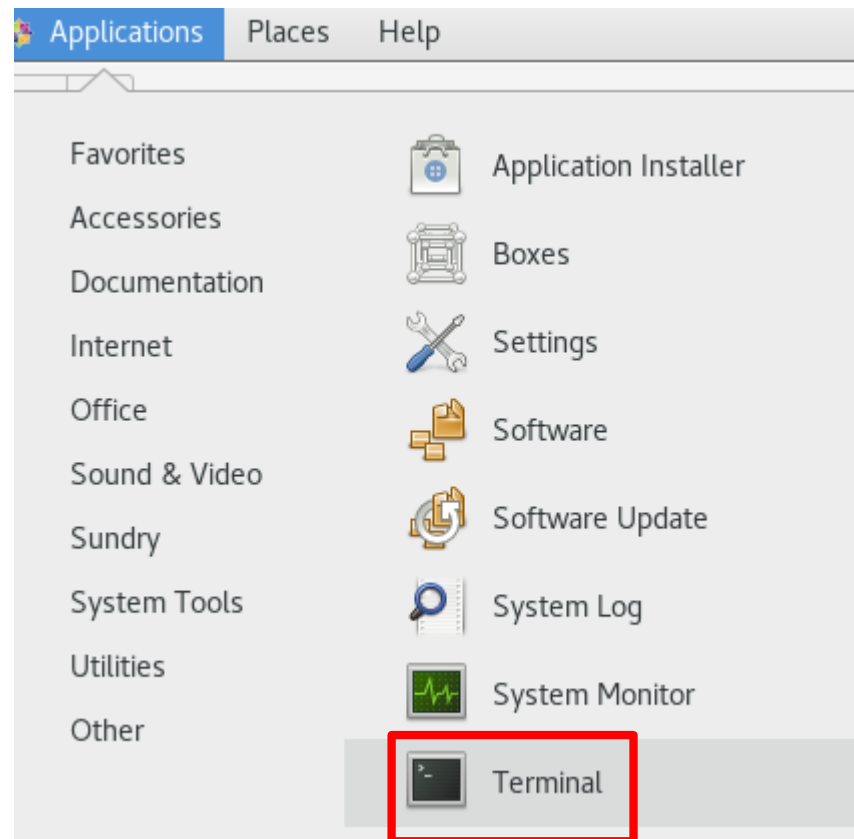
Cancel

Preview changes

Save rules

DCV GUI Access

- In your browser, access <https://<instanceIP>:8443>
- Launch terminal

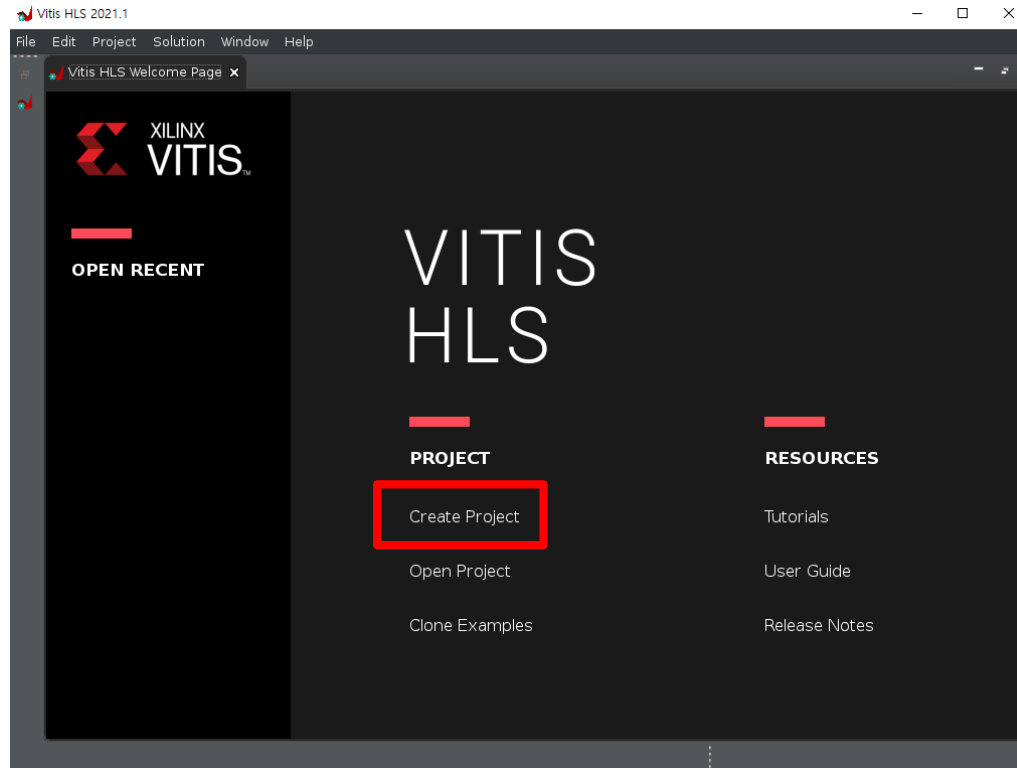


Vitis HLS GUI mode

- Source Vitis HLS

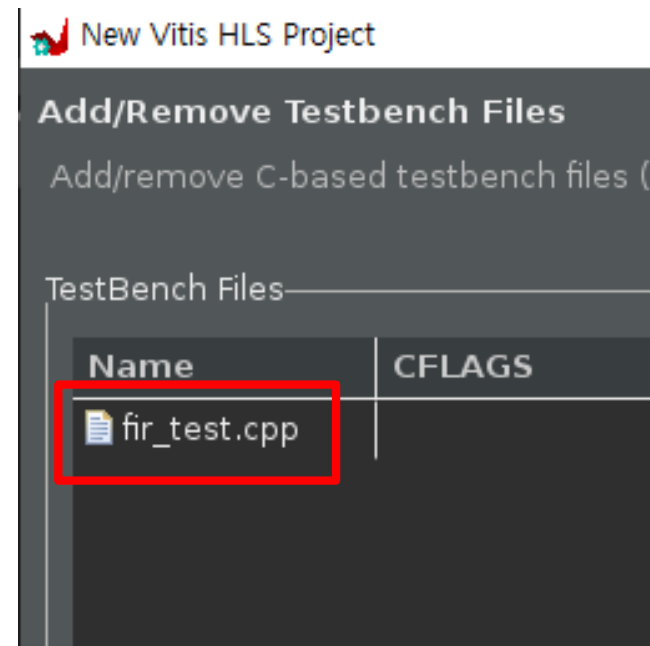
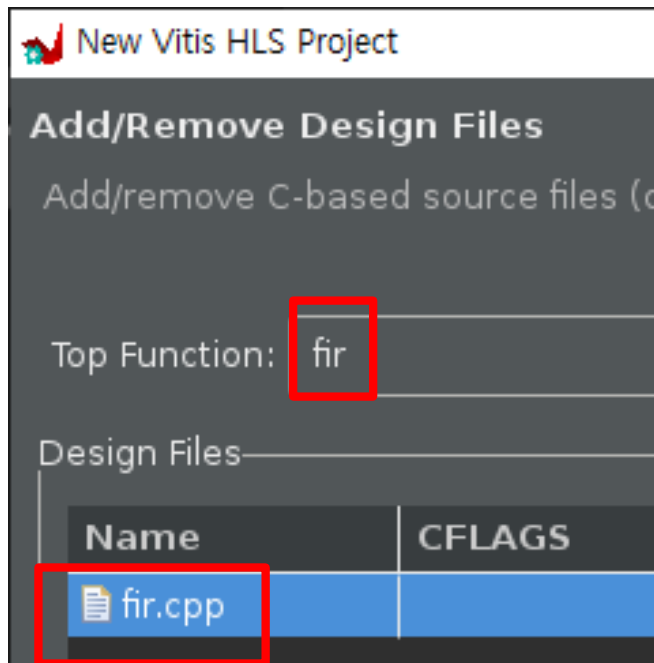
```
source vitis setup.sh  
source /opt/Xilinx/Vitis_HLS/2021.1/settings64.sh
```

- Type “vitis_hls” on your AWS linux console and create a new project



Import FIR source files

- Download FIR filter source code at:
<https://pp4fpgas.readthedocs.io/en/latest/project1.html>
 - Extract files from the “fir11” folder
 - Copy & paste Figure 2.1 code into “fir” function of fir.cpp



Solution Configuration

New Vitis HLS Project

Solution Configuration

Create Vitis HLS solution for selected technology

Solution Name:

Clock

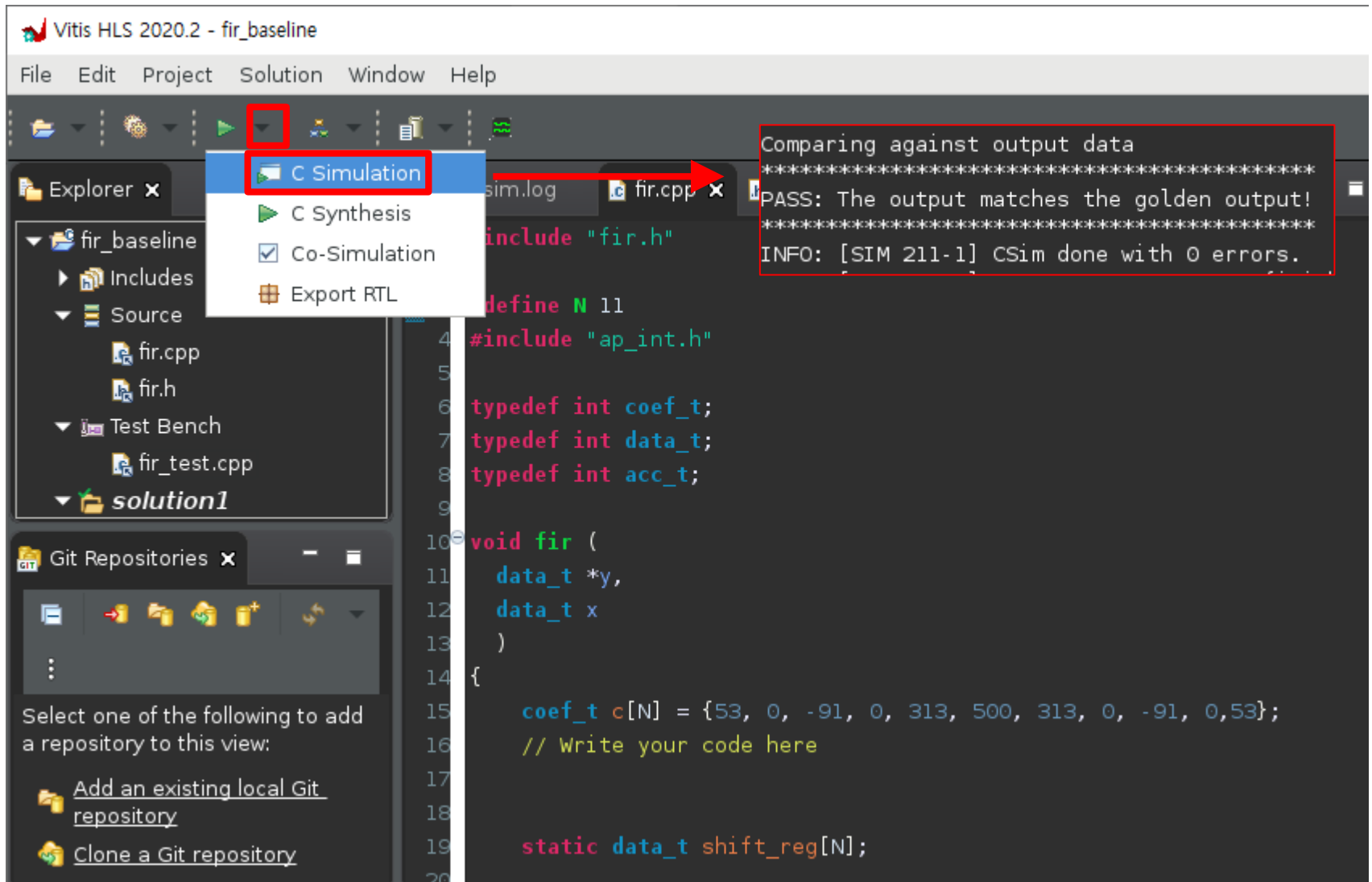
Period: Uncertainty:

Part Selection

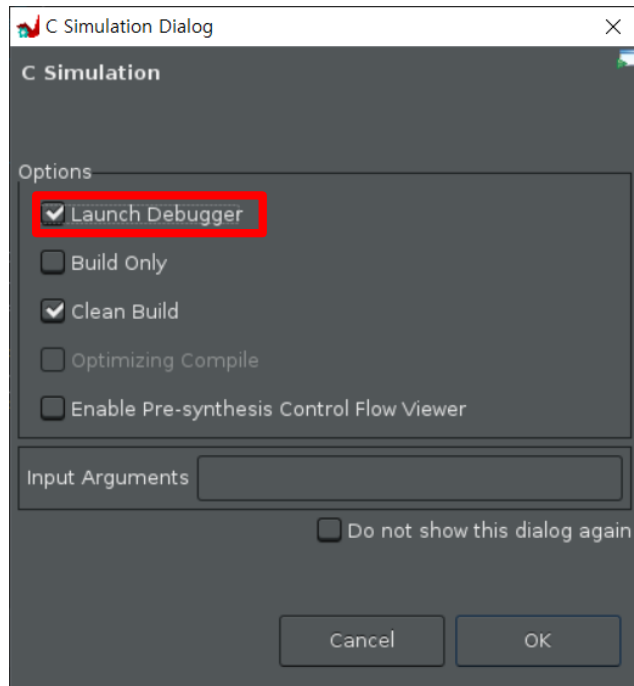
Part: **xcvu9p-flga2104-1-e** (May change to xcvu9p-flgb2104-2-i)

Flow Target

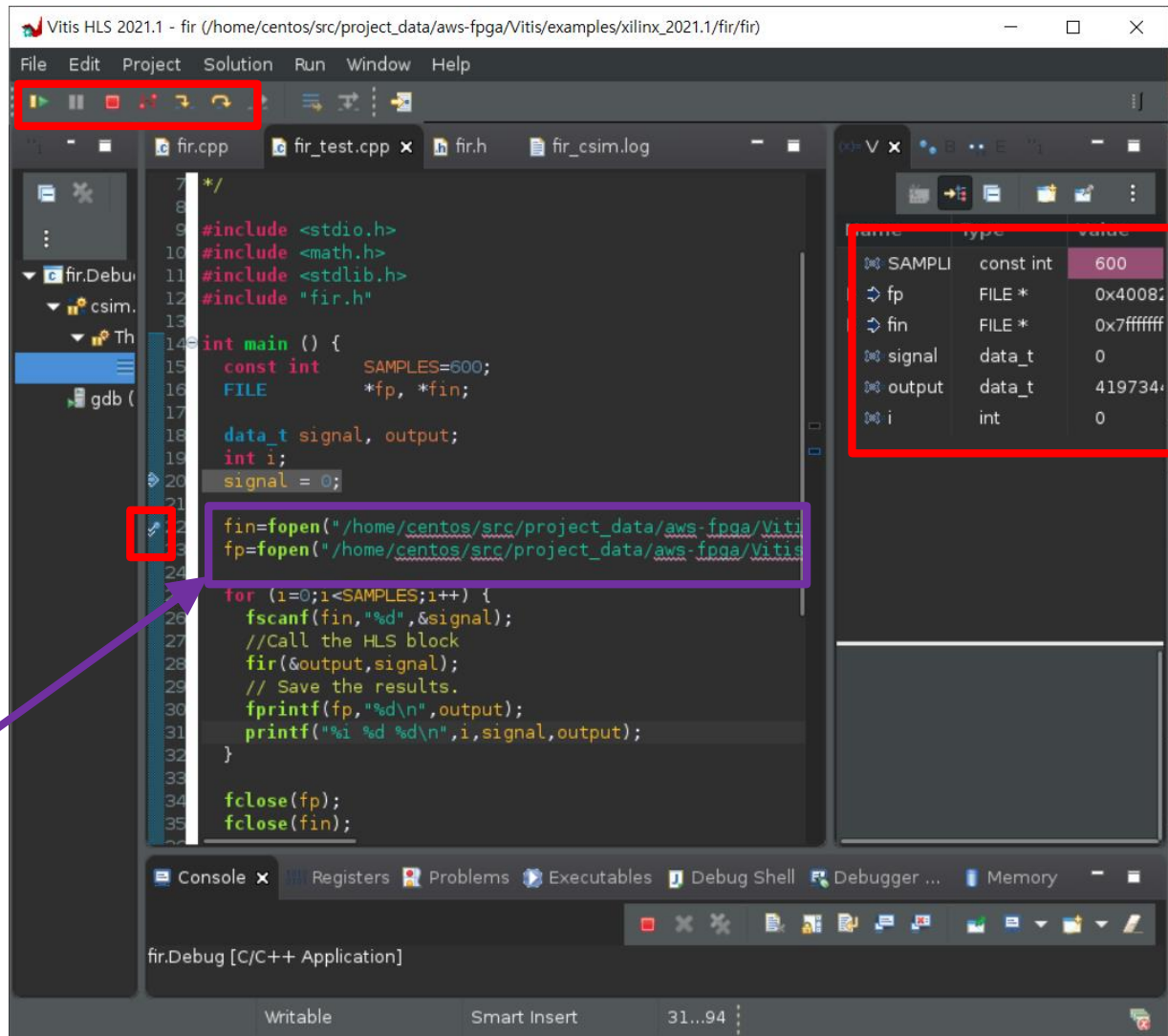
C Simulation



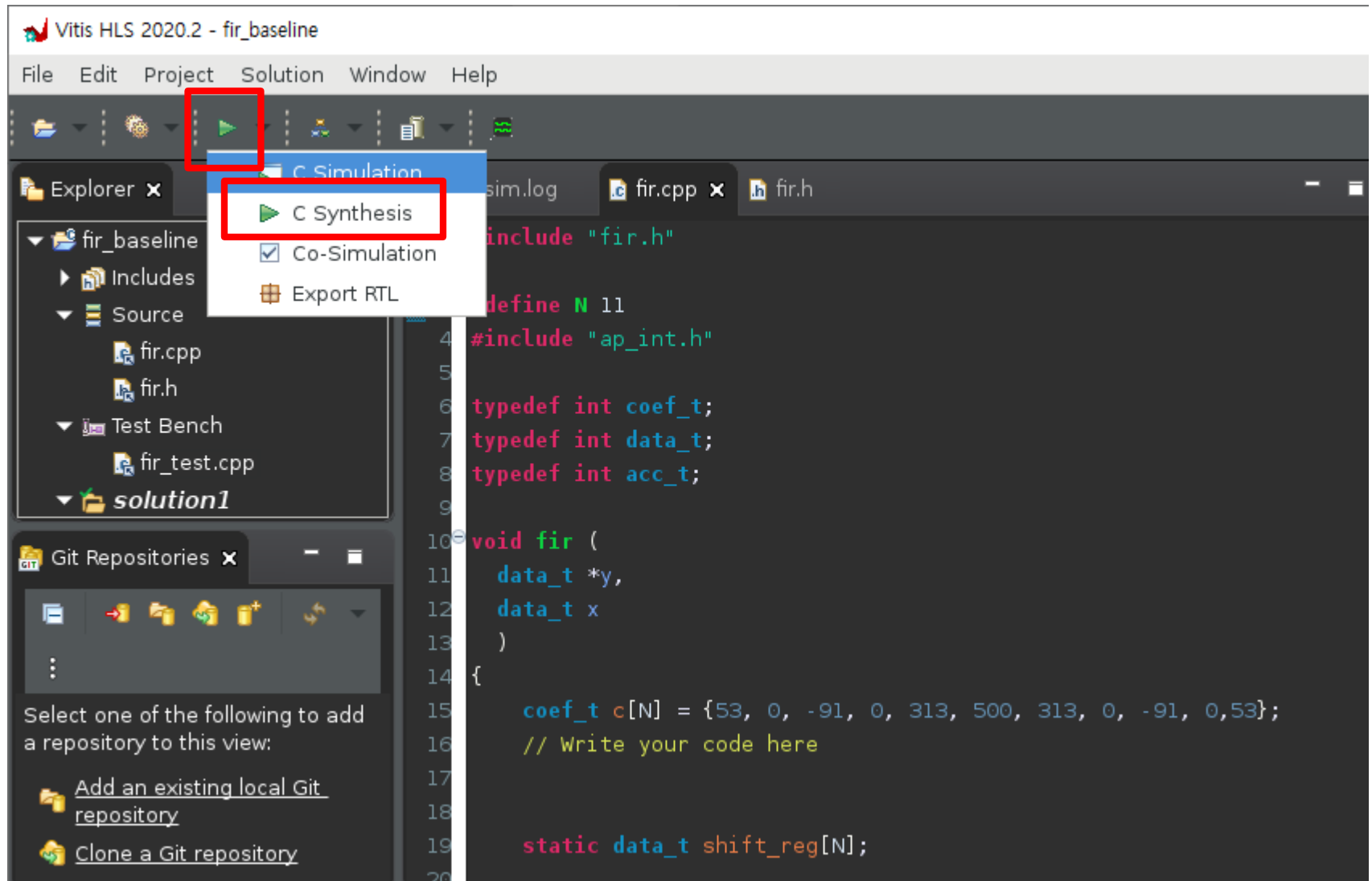
C Simulation – Launch Debugger



Set absolute file path if
fopen causes seg fault



Synthesize your source file



2.4 Calculating Performance

- Execution time (sec) = clock cycles * clock period
= clock cycles / clock freq
 - Default clock freq in AWS F1 : 250MHz

Vitis HLS Synthesis Report

- fir.cpp:

fir_test.cpp: (SAMPLES=600)

```
for (i=0;i<SAMPLES;i++) {  
    fscanf(fin,"%d",&signal);  
    //Call the HLS block  
    fir(&output,signal);  
    // Save the results.  
    fprintf(fp,"%d\n",output);  
    printf("%i %d %d\n",i,signal,output);  
}
```

Project: fir

Timing Estimate

Target	Estimated	Uncertainty
4.00 ns	2.893 ns	1.08 ns

Performance & Resource Estimates

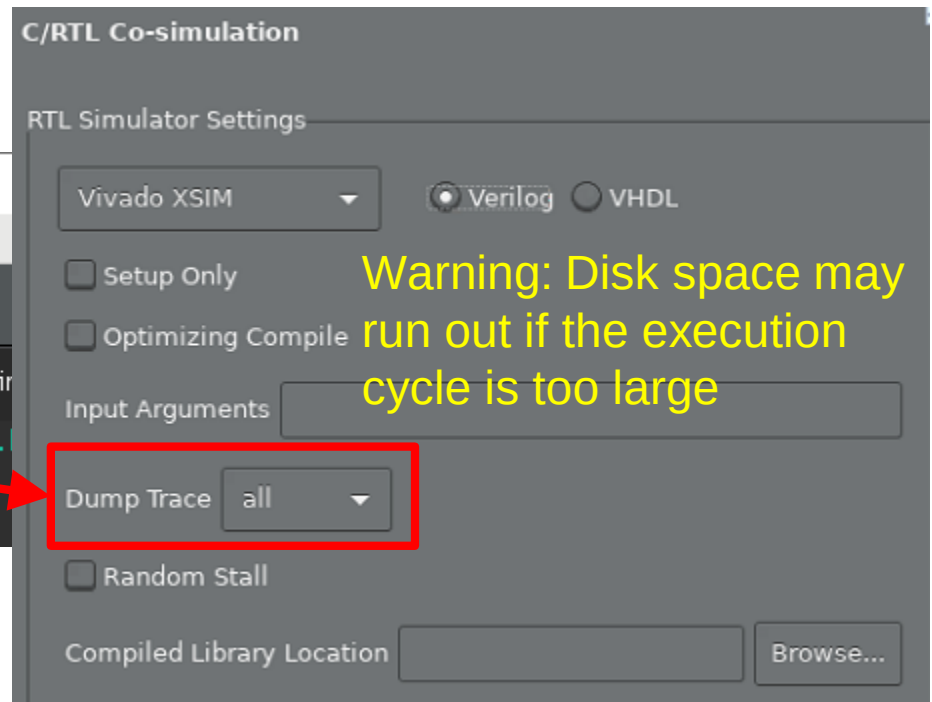
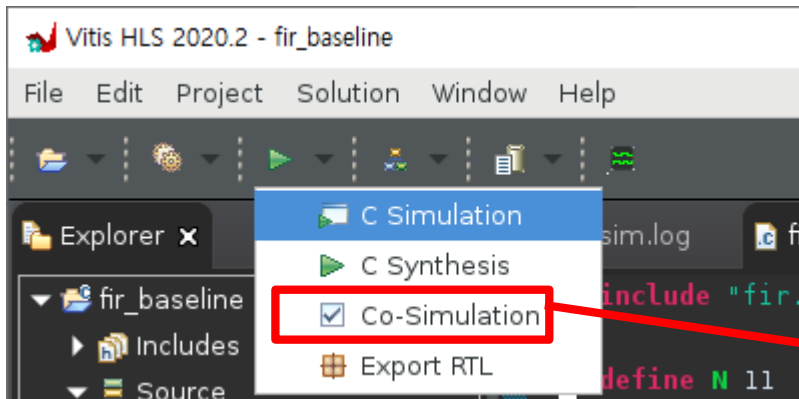
Modules & Loops	Issue Type	Violati	Diste	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined
▼ fir	ii II Violation			-	27	108.000	-	28	-	no
Shift_Accum_	ii II Violation	Memor 1		-	24	96.000	5	2	11	yes

Baseline performance:

600 samples * 28 cycles = 16,800 cycles

Vitis HLS Co-simulation

- Performs C simulation for testbench file (fir_test.cpp), RTL simulation for kernel file (fir.cpp)
 - RTL (register transfer level) simulation : (almost) cycle-accurate simulation
- Running cosim:



Co-sim Result

Project: fir Target device:

Status: **Pass**

Cosim Options

Tool: Vivado XSIM RTL: Verilog

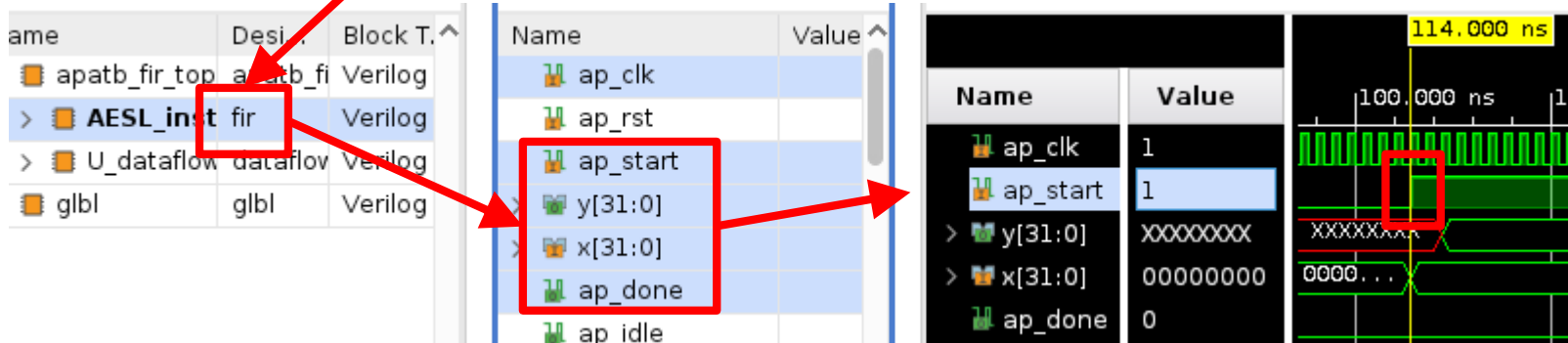
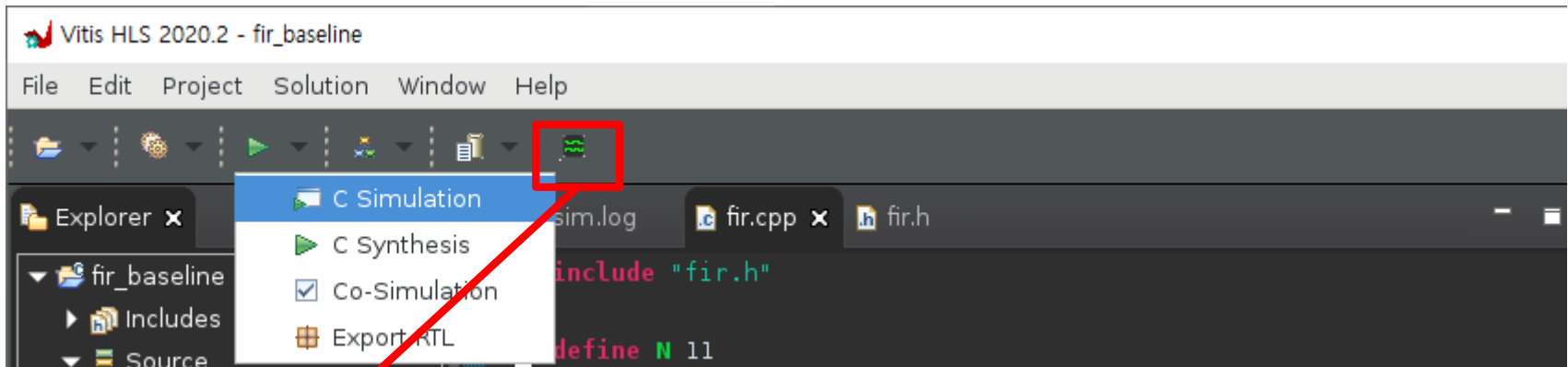
Dump Trace: all

Performance Estimates

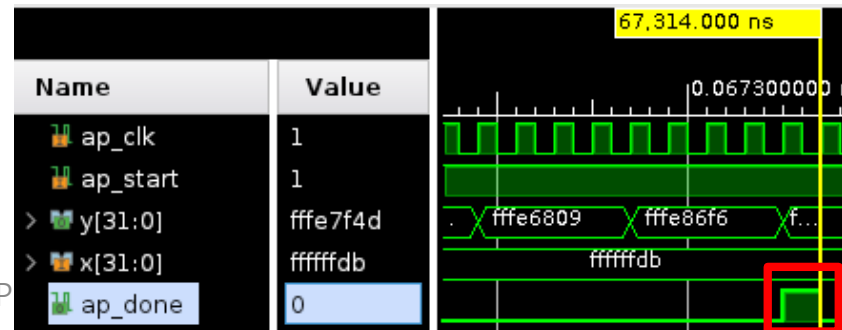


Modules	Avg II	Max II	Min II	Avg Latency	Max Latency	Min Latency
▼  fir	28	28	28	27	27	27
 Shift_Accum_Loop	2	2	2	5	5	5

Waveform Viewing



Overall cycles
 = 67,314 – 114 ns
 = 67.2us
 (16,800 cycles)



Summary

- Have seen a working, synthesizable HLS code for FIR filter
- FIR filter is composed of multiply, add, and register
- Enabled Vitis HLS GUI interface for FIR synthesis and simulation
- C simulation quickly verifies the correctness of a design
- Co-simulation enables waveform viewing and produces cycle-accurate result
- Performance can be expressed in seconds or cycles